

Failure Handling, and Three Mechanisms That Had Never Fired

UM-CYDOS-019

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-CYDOS-019	1.0 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

The kernel had three mechanisms for surviving its own failure: stack guards, a panic handler, and a hang detector. Two of the three had never been observed doing anything, and the third had been silently broken by the second.

This report covers making all three enforce rather than report, and giving each a way to be triggered deliberately. It also records a defect found while building that test harness, which is the most consequential finding here: **the UART receive path was one byte behind, so the interactive shell had never worked from a terminal.**

The unifying failure is not technical. Each of these was verified by observing that it *existed* — a guard word was written, a banner was printed, a watchdog register was armed — rather than by observing it *work*.

2. Recovery and evidence are in conflict

trigger	condition	response	goal
hang	kernel stops making progress unexplained — nothing to read	watchdog resets	recover
fault	illegal instruction, bad pointer explained — the report IS the value	panic, halt, disarm wdt	preserve
guard	stack wrote past its base explained, and bounded at the switch	panic, halt, name the task	preserve

The distinction is whether the kernel can say *WHY* it stopped. It can not explain a hang, so recovery is the only useful response. It can explain the other two, and resetting would destroy the explanation — which it did, until measured.

Figure 1. *How the kernel stops, by whether it can explain itself. Recovery and evidence are in direct conflict, and the watchdog silently won until the conflict was measured.*

The hang detector added in UM-CYDOS-009 §8 feeds on the scheduler switching between distinct tasks. That is the right signal for a hang. It is also exactly what a **deliberately halted** kernel looks like, and `panic.c` ended with:

```
for (;;) {
    /* Deliberately spin rather than reset: a reset loop would scroll the
       * evidence off the terminal. */
}
```

Two correct decisions, taken months apart, that combine into a wrong one. The comment states the panic handler's entire purpose, and arming the watchdog defeated it without touching that file.

2.1 Measured, not assumed

The obvious move is to disarm the watchdog in the panic path. The less obvious one is to first confirm there is anything to fix — the interaction depends on whether the timer interrupt still fires with `PS.EXCM` set, which is a question about silicon rather than about this code.

With the watchdog left armed and `fault` invoked:

```
*** KERNEL PANIC ***
exccause : 0 (IllegalInstruction)
epc      : 0x40083fd0 <- faulting instruction

halted. reset the board to continue.

rst:0x7 (TG0WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
[task 0 entered]
```

The report is produced and then destroyed about three seconds later. With `watchdog_disarm()` in the panic path, output stops at `halted.` and stays there through a fourteen-second capture.

This measurement mattered. Had the tick continued to fire during a panic, the fix would have been wrong *and* would have hidden a worse problem — a scheduler switching away from a faulted context and resuming normal operation on it. The result rules that out: no reporter output appears after the panic banner in either configuration, so scheduling genuinely stops.

2.2 The resulting rule

The distinction is whether the kernel can say **why** it stopped:

- it cannot explain a hang, so recovery is the only useful response
- it can explain a fault or a broken guard, and a reset would destroy the explanation

`watchdog_disarm()` exists solely to express that. It is not a general-purpose control, and the header says so.

3. Stack guards: reported, not enforced

`task.c` filled every stack with a pattern and planted a guard word at its base from the beginning. `task_stack_headroom()` was written to walk that pattern. It was never called — both `high_water` figures anywhere in the tree were the *heap*'s.

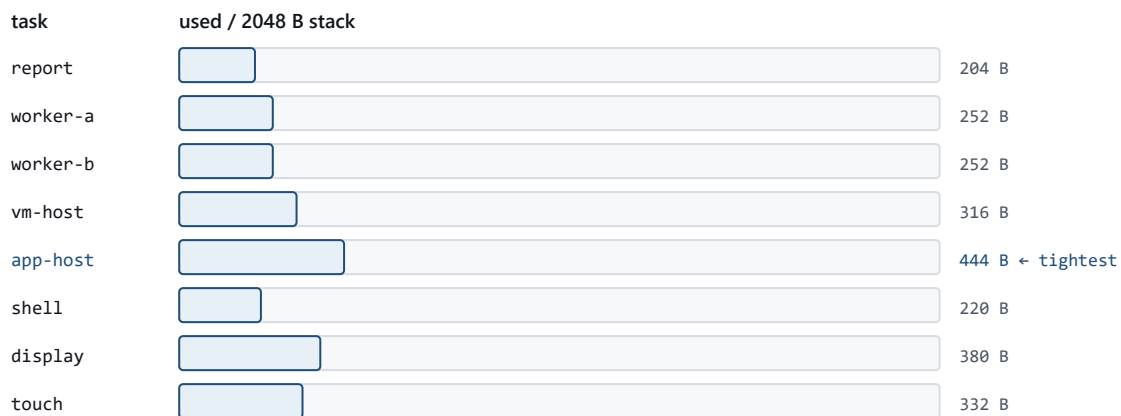
Three separate weaknesses, each individually reasonable:

1. **Coverage.** `kmain` checked guards for `report`, `worker-a` and `worker-b` — three of eight. The display task, which carries the raycaster's call chain, was not among them.

2. **Timing.** The check ran in the reporter, roughly every two seconds. A guard word is broken *after* the damage; a check that runs seconds later reports a corruption whose cause is long gone.
3. **Response.** A break printed `BROKEN` beside the telemetry and execution continued — scheduling tasks whose stacks had already written into a neighbour's, and printing numbers that were by then meaningless.

Checking now happens in `task_schedule()` : all eight tasks, every switch, and a break calls `kernel_panic_msg()` . Eight word loads per tick is not a cost worth weighing against continuing to run on corrupted memory.

3.1 The margins, which were never known



Every task keeps at least 78% of its stack. The worst is app-host, not display — which is where the deepest call chain was assumed to be. Before this, headroom was never measured and five of the eight guards were never checked.

Figure 2. Measured stack use across all eight tasks. The tightest is not the one that was assumed, which is the argument for measuring all of them.

id	name	free B	of 2048			
0	report	1844	4	app-host	1604	<- tightest
1	worker-a	1796	5	shell	1828	
2	worker-b	1796	6	display	1668	
3	vm-host	1732	7	touch	1716	

Every task retains at least 78% of its stack; the worst uses 444 B of 2048. The 2 KB figure in `task.h` was a guess, and it turns out to be a comfortable one.

The tightest task is `app-host` , **not** `display` — which is where the deepest call chain was assumed to be when deciding which three tasks were worth checking. That assumption picked the wrong tasks to watch, and is the argument for measuring all of them rather than the interesting-looking ones.

4. The shell had never worked

While building a harness to send `fault` over the serial line, `mem\r` echoed `mem` and did nothing. `mem\r\n` worked.

That difference is diagnostic. If `\r` triggered execution, then `\r\n` would trigger twice — the second on an empty line, which returns immediately. It working *only* with both bytes means the terminator was not being

consumed at all until something followed it.

Confirmed directly:

```
send "mem\r", wait 2.5 s  -> reply BEFORE the extra byte: no
send " ",      wait 2.5 s  -> reply AFTER  the extra byte: YES
```

The receive path ran exactly one byte behind. `uart_getc_nb()` read the RX FIFO through the APB address at `0x3FF40000`; it now reads the AHB alias at `0x60000000`, and `mem\r` replies immediately.

4.1 Why this is the most important finding here

The shell has existed since UM-CYDOS-013 §4. In every session since, it has been unusable by a person at a terminal — you press Enter and nothing happens, because **Enter is itself the byte left waiting**. The next keystroke would deliver it, so the symptom is not silence but a one-command lag, which reads as "the board is slow" rather than "the input path is broken".

It survived for two reasons, and both are the same mistake:

- every automated test drove it from a sender emitting **both** CR and LF, which is the one input shape that masks the defect
- the shell was verified by observing that its **banner printed**, not by observing that a command **ran**

A banner proves the task was created and the transmit path works. It says nothing whatsoever about receive. The same substitution — a startup artefact standing in for the behaviour it introduces — is what let the guard checks and the panic handler go unexercised for as long as they did.

5. Triggering failure on purpose

Three shell commands exist solely to make these paths reachable. `hang` predates this work; `fault` and `smash` are new.

Command	Induces	Expected
<code>hang</code>	masks interrupts, spins	watchdog resets the board
<code>fault</code>	executes <code>ill</code>	panic, halt, evidence retained
<code>smash</code>	clobbers the running task's guard word	panic at the next switch, naming the task

Verified on hardware:

```

smash -> *** KERNEL PANIC ***
        reason  : stack guard overwritten
        detail   : 5                      (task 5 is the shell)

fault -> *** KERNEL PANIC ***
        exccause : 0 (IllegalInstruction)
        halted.   still halted after 14 s

hang -> rst:0x7 (TG0WDT_SYS_RESET)      recovered, rebooted

mem\r -> heap free=71792 ...             no trailing byte needed

```

`smash` correctly identified task 5, which is the shell — the task that invoked it. That is a stronger result than a bare panic, because it shows the scheduler attributing the break to the right task rather than to whichever it happened to be examining.

These are deliberately not compiled out. A destructive command in a development shell is a smaller risk than a recovery path nobody can reach to test.

6. Metrics

Quantity	Value
Tasks with guards checked, before → after	3 → 8
Guard check frequency, before → after	~2 s (reporter) → every context switch
Guard check cost	8 word loads per tick
Stack headroom, worst case	1,604 B free of 2,048 (app-host)
Stack headroom, best case	1,844 B free of 2,048 (report)
Minimum margin across all tasks	78%
UART rx lag, before → after	1 byte → 0
Sessions the shell was unusable interactively	every one since UM-CYDOS-013
Failure paths now triggerable on demand	3

7. What this does not establish

- **No pre-emptive overflow detection.** A guard word is found *after* it has been overwritten. The margins in §3.1 say that is not close to happening, but the mechanism is still a post-mortem, not a barrier. A guard page would be a barrier; this hardware has no MMU to provide one.
- **No wild-pointer protection.** `task.h` has always said this: a native task can corrupt any other. Guards catch the common case — growth past a stack's own base — and nothing catches a stray write into the middle of a neighbour.
- **The panic reason is not persisted.** UM-CYDOS-018 built a record that survives a power cycle, and the panic path does not use it. A fault with no serial attached is still invisible.

- **The panic reason never reaches the screen.** The device is standalone and has a display; a panic currently produces a frozen panel.
- **No stack sizing per task.** All eight get 2 KB regardless of measured use. The figures in §3.1 would support trimming several, but nothing depends on reclaiming that memory yet.
- **The AHB alias is validated behaviourally, not from documentation.** The measurement establishes that the APB address lags and the AHB alias does not. It does not establish *why*, and no erratum has been cited here to support it.

8. References

- UM-CYDOS-009 §8 — the hang detector this reconciles with the panic path
- UM-CYDOS-013 §4 — the shell and its polled UART receive, verified by its banner
- UM-CYDOS-018 §5 — the previous instance of a harness lying about its own results
- UM-CYDOS-017 §6 — the standing rule on instruments reporting themselves invalid
- `kernel/task.c` — `task_stack_broken()`, `task_stack_tightest()`, checked in `task_schedule()`
- `kernel/panic.c` — `halt_forever()`, `kernel_panic_msg()`
- `kernel/uart.c` — `UART_FIFO_AHB_REG` and the reasoning above it